EEMEM00018 – IoT System Prototyping

Coursework Assignment

This document outlines the coursework for EEMEM0018 IoT System Prototyping unit. You are required to carry out **two design challenges (see Section 3 and Section 4)** in groups of three members (max) and submit **a 12-page report** for assessment **before 1PM on 28th April, 2026**.

Please note that while the core software artefact will be assessed as a group submission, **individual marks will be awarded based on demonstrated personal contribution, peer marking, technical understanding, and reflection**. Each student must take responsibility for a clearly defined task or subsystem within the application and explain and justify their individual design decisions in the report be able to explain and justify their design decisions during individual assessment activities. The final individual mark will be calculated based on the individual contributions described in the report.

1. What should be submitted

1. **Technical Report (Blackboard Submission):** The final unit assessment is a 12-page technical report, formatted in 11-point Arial or Calibri with 2cm margins, representing 100% of the grade on a 20-point scale. Allocate 3–4 pages to Design Challenge 1, followed by 4–5 pages for Design Challenge 2. Provide information about your code quality for example memory footprint as reported by the build system.

   The final section, limited to 3 pages total with a maximum of one page per member, must detail individual reflections on technical decisions, the management of team disputes, and a design trade-offs in the work that you personally carried out. While appendices may contain supplementary data, only the first 12 pages will be evaluated, and final individual marks will be adjusted via the Peer Assessment Factor to reflect actual contribution levels.

   One member from the group can submit this report.

2. **Source Codes (Blackboard Submission):** complete project file archive for design challenge 1 and 2. One member from the group can submit this report.

3. **Peer assessment form (MS Form Submission)**: Failure to submit the peer marking form will result in a default contribution score of 100%, which may significantly impact your final individual grade if your peers have rated you differently. Please use the form: EEMEM00018 - IoT System Prototyping Peer Marking Form – Fill in form

2. Using AI in Assessment

In this assessment, students may make use of Category 2 (Minimal) generative AI tools, as defined in the University of Bristol's guidance on using AI in assessment. Under this category, students are permitted to use AI only for limited, surface-level support such as spell-checking

and grammar-checking, but not for rewriting, generating, or substantially editing content. This approach aligns with the University's default position on AI use and helps maintain academic integrity by ensuring that students' submitted work continues to demonstrate their own understanding and skills. Clear adherence to Category 2 ensures that learning outcomes are met while still allowing students to benefit from basic assistive technologies

3. Design Challenge 1: High-Reliability Thermal Monitor and Controller

Build a multi-threaded system on the nRF54L15 DK that monitors temperature using LM335 temperature sensor, detects hardware faults, and manages alarms. Your team are the lead application architects and therefore the choice of approach and data presentation is your choice, but your design will be judged on resource efficiency, and stability.

The Requirements: You must implement at least four distinct threads that handle following responsibilities:

1. **Data Acquisition:** Sample data at the rate of 100 ms. Detect and handle ADC errors (e.g. impossible or out-of-range voltages)
2. **Logic Engine:** Compute a 1-minute average. Trigger a 50 ms LED blink if the average temp is larger than 28°C.
3. **Reporting:** Output a structured status log to the serial console. For example:

```
[TIMESTAMP] Avg: 27.4C | Raw: 27.1C | Mode: NORMAL    | LED: OFF
[TIMESTAMP] Avg: 28.2C | Raw: 28.5C | Mode: WARNING   | LED: BLINKING
[TIMESTAMP] Avg: --.-C | Raw: 0.05V | Mode: FAULT     | LED: SOLID
```

4. **Data communication:** Periodically transmits a BLE advertisement message that contains:
   - The 1-minute average and
   - The system state (**NORMAL** / **WARNING** / **FAULT)**. Note that when the system enters state "FAULT" it may be impossible to transmit BLE advertisements.

How threads communicate and how they are prioritized should be designed according to your judgment. The highest-performing systems will demonstrate:

- **Minimal SRAM Footprint:** Storing a 60-second history of float data in a large array is "expensive." Can you find a way to compute a 1-minute average using only a few bytes?
- **Zero Jitter:** The 100 ms sampling must be 100% precise. If any other tasks running on your device cause a sampling delay, your priority logic is flawed.
- **Thread Safety:** Moving data from the sensor to the brain must be "lossless." If you use global variables without protection, you risk data corruption.
- **Deterministic State Transitions:** Your system must handle the transition between **NORMAL**, **WARNING**, and **FAULT** without ever entering an undefined state.

Students aiming for a higher score should implement the following **Extension Tasks (See the Mark Scheme)** to demonstrate advanced mastery of the nRF54L15 and Zephyr RTOS:

1. **Live Calibration**: Implement a mechanism to update the warning threshold. The choice of mechanism is your decision, it could be a command over the serial line, a button press, or a command over the network, any combination of the above or a different alternative mechanism of your choice. The calibration process must not affect data acquisition from the sensor. How do you ensure the Processing thread sees the new threshold safely while it's in the middle of a calculation?
2. **Drift Detection**: Sensor drift occurs when the electrical properties of the sensor change over time (due to heat, humidity, or component aging), causing the "zero point" or baseline to move. If the 24-hour baseline moves by more than $\pm 2.0^{o}C$ while the device is in a controlled environment, flag the **DRIFT** state. You are encouraged to use memory-efficient algorithms like **Welford's Method** or a **Running Variance** to track this shift without storing 24 hours of raw data.

What to Include in Your Report: This part should be described in a dedicated section between 3 and 4 pages long. It must prove that your system is stable, efficient, and reliable. Use the following four sections in your report for this part:

- RTOS Architecture & Threading: Provide a thread priority table and IPC diagram (e.g., Mutexes, Message Queues) to justify how your design ensures determinism and zero jitter for the 100ms sampling rate.
- Computational Efficiency: Document the specific algorithms used (such as Welford's Method) to achieve low memory usage, proving you minimized the SRAM footprint for long-term averaging and drift detection. You may wish to include and comment on your work's code and memory footprint as reported by the Zephyr build system.
- Live Calibration & Safety: Detail the implementation of your threshold update mechanism, focusing on how you used concurrency control to ensure the system remains stable and thread-safe during live adjustments.
- Systematic Testing: Demonstrate operation through empirical evidence, including timing logs for jitter analysis and serial captures showing successful transitions between NORMAL, WARNING, and FAULT states.

4. Design Challenge 2: Embedded Gesture Recognition with TinyML

In this challenge, you will work with an existing embedded machine learning system based on the Arduino Nano 33 BLE Sense and the Magic Wand example provided in the Harvard_TinyMLx package. A working baseline system is provided and is not assessed. Assessment focuses on your ability to modify, extend, and reason about an embedded ML system through your own code and experiments. Please note that you cannot modify the current dataset split ratio.

4.1. Baseline (Not Assessed)

Enclosed is the Jupyter Notebook for the baseline: UoB-IoT-tb2-MagicWand - Colab
please copy it to your own Google Drive.
The provided baseline is the Magic Wand digit recognition example (0–9):

- Environment: Google colab
- Sensor: IMU (accelerometer + gyroscope)
- Task: digit gesture recognition
- Platform: Arduino Nano 33 BLE Sense
- Model: CNN-based neural network deployed using TensorFlow Lite Micro

You must successfully run the baseline before starting the assessed tasks, but baseline performance itself does not contribute to marks.

4.2. Architecture Exploration

The objective of this task is to understand how model the architecture affects accuracy and memory usage in an embedded ML system. Using the provided training dataset (no personal data at this stage):

1. Design and train at least 5 different model architectures by modifying:
    - number of layers,
    - number of channels per layer,
    - layer width or kernel size.
2. For each model, record:
    - classification accuracy on test dataset on the PC,
    - the final model size (TensorFlow Lite Quantized).
3. Plot and analyse the accuracy vs model size trade-off.

What to include in the report

- A plot showing accuracy vs model size. The range of model accuracy should cover 70% ~ 90%.
- A plot showing accuracy vs model latency on the MCU. Please use the same accuracy range as in the sub-task above. A short explanation of the observed trend.

4.3. Personalisation and Stability Analysis

In this task we you are going to study how personal data affects model behaviour, especially prediction stability on an embedded device. The requirements and the process are outlined below:

1. Select 2 numbers based on your group ID (the 1st number is Your_group_id % 10, and the 2nd one is Your_group_id % 10 + 1).
2. Collect a personal gesture dataset of the two numbers using the given data collection procedure (See the Appendix).
3. Mix your personal data with the original dataset and retrain the model.
4. Compare the baseline model and your personalised model in terms of:
    - PC-based classification accuracy,
    - on-device prediction stability.

5. Define and implement a consistency metric, for example repeating the same gesture 10 times and measuring how consistent the predictions are.

What to include in the report

- Accuracy comparison (baseline vs personalised model).
- Numerical results of the consistency metric and comparison.
- Brief explanation of the results.

5. Extending the System: Adding a New Gesture Class

This task evaluates the scalability of an embedded ML system when new functionality is introduced.

1. Collect gesture data for one additional class (your assigned letter is given by Your_group_id % 26).
2. Extend the classifier from 10 classes (digits 0–9) to 11 classes.
3. Retrain the model and analyse:
    - the impact on digit recognition accuracy,
    - how well the new class is recognised,
    - any confusion between classes.

What to include in the report

- Accuracy results for the 11-class model.
- Numerical results of the consistency metric of the new letter.
- A short discussion on the effect of class extension.

6. The Marking Rubric:

Component	Percentage	Mark Scale					
		0-5	6-8	9-11	12-14	15-17	18-20
Design Challenge 1	35%	1)Single-threaded logic used. No evidence of thread safety or ADC error handling. Design ignores the fundamental constraints of the nRF54L15.	High risk of data corruption due to unprotected globals. Logic transitions are "noisy" or poorly defined. The design lacks the determinism required for an RTOS.	Uses multiple threads, but synchronization is weak (e.g., relies on "luck" or simple delays rather than kernel signals). Core logic works but ignores edge cases (like ADC saturation or sensor disconnect).	Correct transition logic between Normal, Warning, and Fault. ADC sampling logic is decoupled from "heavy" tasks like Serial/BLE logging. Clear separation of concerns across the 4 required threads.	Clear, safe logic for Calibration or Drift detection. Evidence of correct thread synchronization (Mutexes/Semaphores) to protect shared state. Avoidance of heavy buffers; smart use of Zephyr's kernel features.	Implementation of algorithms is mathematically sound. Thread priorities and scheduling are perfectly tuned for zero-jitter sampling. Robust IPC (Inter-Process Communication) and total protection against race condition
Design Challenge 2	35%	1) Fewer than 5 architectures explored; critical records (accuracy/model size) or meaningful plots are missing. 2)	1) 5 architectures are trained, but they show only minor variations with a weak design rationale. 2) Includes	1) Explores at least 5 distinct architectures with a basic trade-off analysis. The 11-class extension is implemented	1) Architecture exploration is systematic, with a clear explanation of the relationship between accuracy and	1) Core tasks and Random Search are fully implemented. The personal dataset is diverse, balanced, and	1)Implements Advanced Search (e.g., Bayesian Optimization) with a coherent discussion about accuracy, model

		Personalization or automated search tasks are absent or fundamentally flawed.	accuracy vs. model size plots, but the discussion lacks technical depth. 3) The personalization experiment and consistency metrics are present but lack the rigou1r or justification required for reliable on-device deployment.	and functional. 2) Conducts a baseline vs. personalized comparison and numerically evaluates the consistency metric. 3) Automated search satisfies the >=92 accuracy constraint, but the accompanying analysis is descriptive rather than evaluative.	model size. 2) Personalization analysis moves beyond raw accuracy to discuss the trade-offs between generalization and stability. 3) The 11-class extension and automated search space are clearly defined, though the technical justification for specific search details lacks sufficient depth.	thoroughly documented. 2) Provides an effective, data-driven comparison between base and personalized models using advanced metrics. Proper reasoning about personalization effects on stability.	size, stability, and scalability. The 11-class extension demonstrates no class interference or "forgetting." 2) Identifies high-level trends such as the correlation between model topology and hardware-specific latency bottlenecks. Synthesis of results is professional and publishable in quality.
Technical Report	20%	**Unsatisfactory:** Lacks original thought or fails to match the actual implementation.	**Poorly organized;** difficult to follow the design logic. Significant gaps in the	**Basic Function:** The report identifies the task and shows a working solution. **Documentation:**	**Completeness:** All required sections (Methodology, Results, Individual	**Clarity:** Well-structured with a logical flow from requirements to testing. **Analysis:** Correct	**Technical Depth:** Advanced use of system block diagrams and state machine

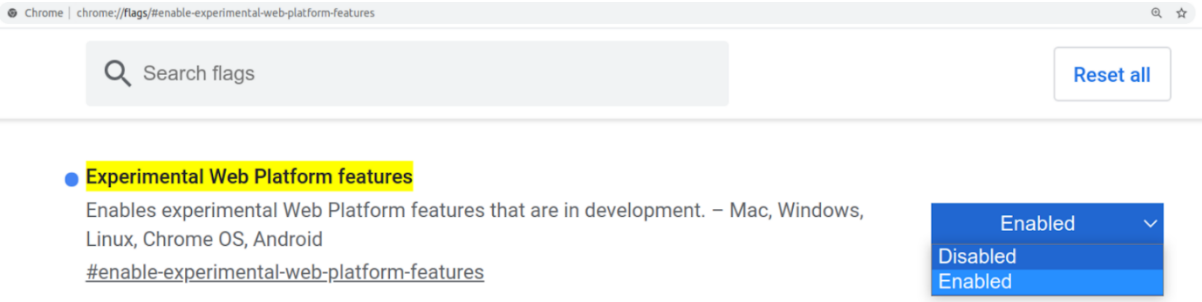
		Missing the individual reflection or peer assessment elements entirely.	methodology. Fails to address core requirements like thread safety or signal conditioning limits.	Relies heavily on raw code or raw data without sufficient explanatory prose.	Contributions) are present. **Accuracy:** Diagrams are mostly correct, though perhaps lacking minor detail.	interpretation of data (e.g., noise floors, sampling jitter, or thermal drift).	transitions. **Justification:** Clear evidence of why specific components/algorithms were chosen over alternatives.
Teamwork Reflection	10%	**Missing/Incoherent:** Section is absent or appears to be generic filler text with no relevance to the actual implementation.	**Under-developed:** Only a few sentences provided. Fails to link personal effort to the final outcome. No mention of the other team members or the integration phase.	**Descriptive Only:** Mostly a "diary" of what was done (e.g., "I wrote the ADC code"). **Generic:** Reflection is vague (e.g., "I learned a lot about Zephyr"). No mention of specific engineering challenges or team friction.	**Technical Accuracy:** Describes their work accurately but lacks a deep "Lesson Learned" or "What I'd do differently." **Collaboration:** Mentions group meetings and task division without analyzing the efficiency of the workflow.	**Technical Insight:** Connects personal tasks to core requirements **Honest Evaluation:** Discusses what went well and what didn't, using technical evidence. **Contribution:** Clearly defines their role in the group's success.	**Deep Analysis:** Evaluates the *why* behind technical trade-offs. **Self-Correction:** Identifies specific mistakes and explains the engineering logic used to fix them. **Team Dynamics:** Analyzes how individual work packages affected the whole system's stability.

Appendix

How to collect a personal gesture dataset:

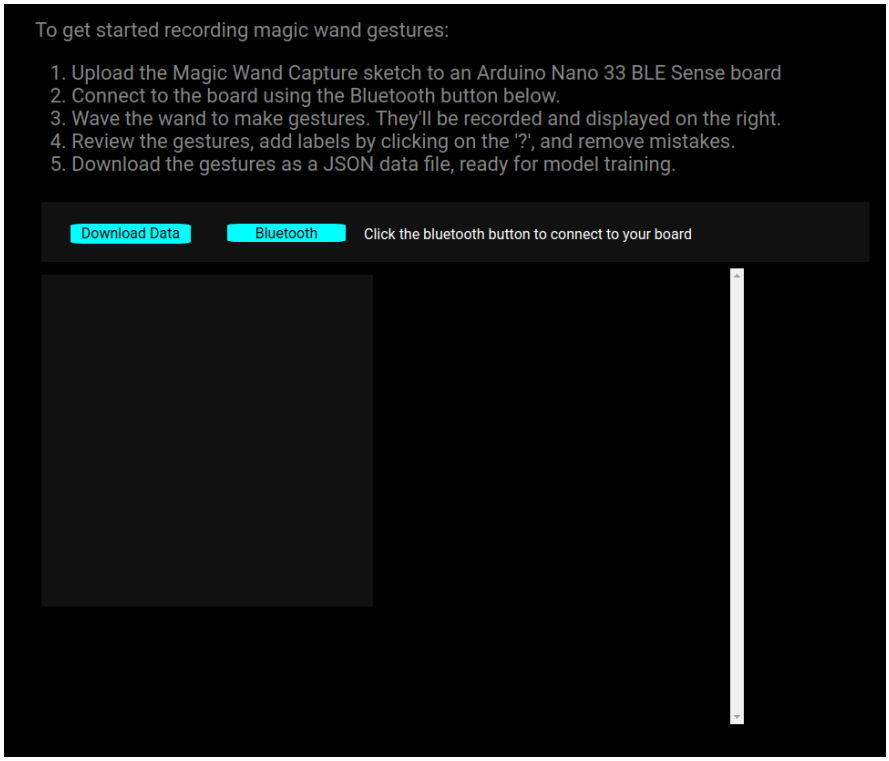
(1)Open up your browser and navigate to: https://tinyml.seas.harvard.edu/magic_wand

(2) If you see a warning that says: Error: This browser doesn't support Web Bluetooth. Try using Chrome. If you are not using Chrome, we suggest you switch to Chrome. If you are already using Chrome, then navigate to the following and make sure to “enable” the “experimental web platform features.” We have found this step to be necessary on older versions of Chrome and users running the Linux operating system. chrome://flags/#enable-experimental-web-platform-features.



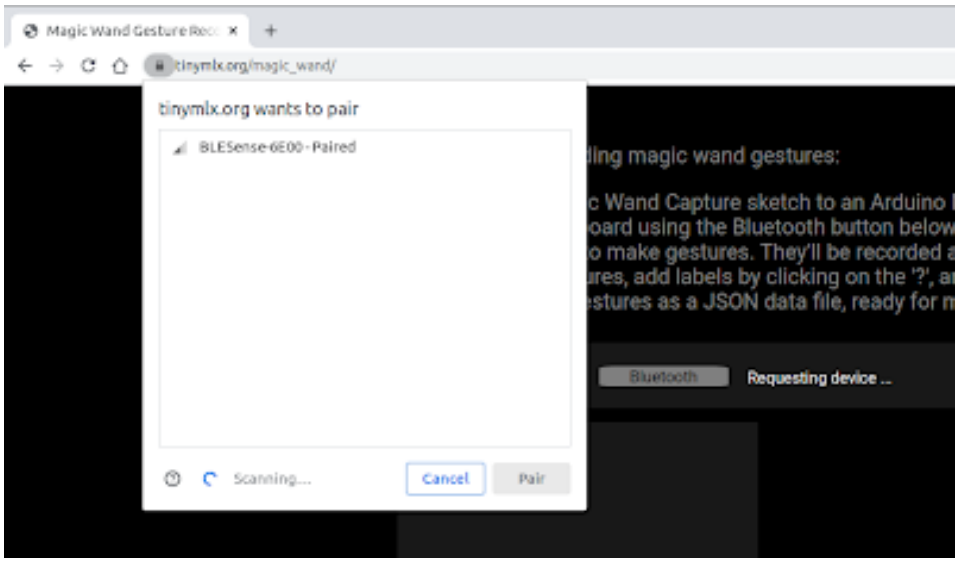
If this problem persists, make sure you are navigating to https://. For some reason, if you do not use the secure protocol, then the web Bluetooth will not work.

(3) You should then arrive at a webpage that looks something like the following:



(4) You’ll then need to connect your device over Bluetooth. To do that, simply click the blue Bluetooth button and a pop-up will appear asking to pair. Select your BLE Sense, which should be called something like “BLESense-XXXX” where XXXX will vary, and click the pair button. **Do note that**

the course staff has found that sometimes you have to repeat this step twice. Once you are connected, the Bluetooth button will turn green.

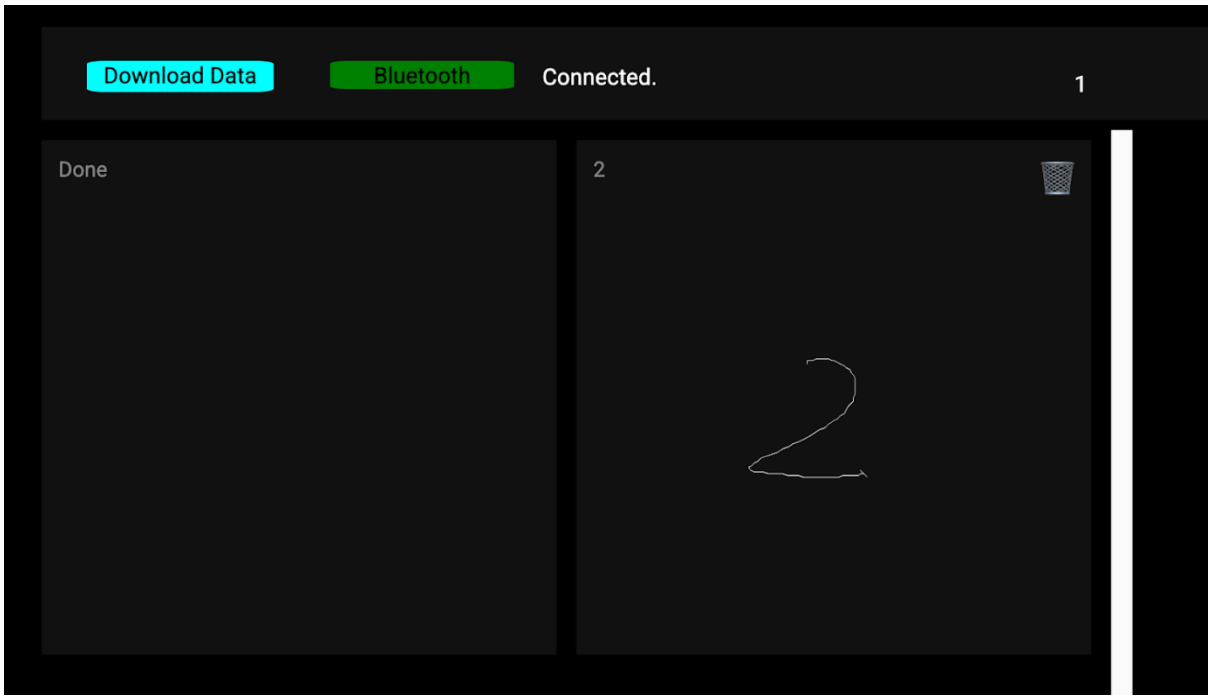


(5) Once your Arduino is paired, it's time to record some gestures. You'll notice that every time you move the Arduino around and then stop, a new gesture is recorded. This is because the gestures are automatically split up by times when the wand is kept still. These pauses act like spaces between words, and so when you've finished, a gesture you should stop moving the wand, so that it ends cleanly. **Note that the direction of the gesture matters (e.g., a clockwise circle is different from a counterclockwise circle)!**

These gestures you are drawing will start to show up in the list on the right side of the screen. You can look at the shapes shown there to understand whether the gestures came out cleanly. **A good rule of thumb is that if you can't tell what the gesture is by looking at it, then a model will have a hard time recognizing it too.**

If you want to delete a recording, simply click the trashcan icon at the top right of each gesture recording (You may need to delete a lot of spurious recordings that were made as you moved the Arduino into position between each gesture).

Also, make sure to label all of your gestures for training. You can label each recording by clicking on the question mark at the top left of each gesture and typing in your label. For example, the screenshot below shows the label "2" added to the gesture of the number "2" recorded by the course staff. **Make sure you label all of your gestures and note that case matters!** Some students have noted that it can be easier to create one gesture per recording session in order to make sure all of the gestures are labeled identically but/and the staff has also gotten it to work with multiple gestures per recording session.

The staff has found that collecting ~20 examples each of 2-3 different gestures often will be enough data to successfully train a moderately decent magic wand application (i.e., it will work often for you may not generalize to other users). To help you keep track of how many gestures you recorded, there is a number in the top right of the screen (e.g., the number 1 as shown in the image below).

Finally, you can upload multiple JSON files to the training script so don't feel pressured to do all of your gesture recordings in one shot!